



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验七：内存管理与用户堆

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 完善物理帧分配器，使已经释放的物理帧能够被后续分配复用。
2. 理解进程虚拟内存中代码段、栈、堆和页表的所有权，正确释放退出进程占用的页面。
3. 统计进程虚拟内存和内核物理内存的使用情况，并在进程列表中显示。
4. 实现内核栈按需增长，理解页故障处理与页面权限之间的关系。
5. 实现 `brk` 系统调用、用户堆扩缩和用户态动态内存分配。

二. 实验环境

项目	配置
操作系统	YatSenOS / YYOS
开发语言	Rust (<code>no_std</code> 、 <code>no_main</code>)
目标架构	x86_64
引导环境	UEFI + OVMF
虚拟机	QEMU
构建工具	Cargo 、 Make
报告工具	Typst

三. 实验内容概述

本实验从“用户程序的内存释放”开始，完成了实验七后续必做任务。实现后的内存管理路径如下：

```

Bootloader
|-- 加载内核 ELF，记录映射页范围
|-- 仅映射内核栈初始页面
`-- BootInfo(kernel_pages, memory_map, ...)
    |
Kernel    |-- BootInfoFrameAllocator：新帧 + 回收帧
            |-- ProcessVm：代码段 + 栈 + 堆 + 页表上下文
            |-- Page Fault：用户栈或内核栈按需增长
            `-- sys_brk：调整用户堆边界
    |
User library    `-- brk_alloc 全局分配器

```

物理帧分配器负责“帧是否可再次使用”，`ProcessVm` 负责“一个进程拥有哪些虚拟内存区域”，页表上下文则通过 `Arc` 表达 `fork` 后父子进程共享地址空间的生命周期。

四. 实验原理

1. 帧回收

Bootloader 提供的可用物理帧迭代器是惰性的，适合顺序分配，却不能把已经取出的帧重新放回原迭代器。因此分配器增加 `Vec<PhysFrame>` 作为回收栈：

```

pub struct BootInfoFrameAllocator {
    size: usize,
    frames: BootInfoFrameIter,
    used: usize,
    recycled: Vec<PhysFrame>,
}

```

分配时优先 `pop` 回收帧，回收集合为空时再从启动内存图迭代器中获取新帧；释放时将帧 `push` 到回收集合。`used` 统计从启动内存图中取出的帧数，当前仍在使用的帧数可表示为：

$$\text{live_frames} = \text{used_frames} - \text{recycled_frames}$$

使用 `Vec` 时每个记录约占 8 字节，空间效率低于位图，但实现简单，适合本实验规模。

2. 进程虚拟内存的所有权

用户进程的内存占用由三部分组成：ELF 映射、用户栈和用户堆。本实验忽略页表本身的统计，但在进程退出时仍会释放页表页面。

ELF 加载函数返回实际映射的页范围，`ProcessVm` 保存这些范围并累计字节数。栈和堆各自记录当前范围，并提供 `memory_usage` 与 `clean_up`。因此：

$$\text{process_memory} = \text{elf_memory} + \text{stack_memory} + \text{heap_memory}$$

`fork` 后页表上下文由 `Arc` 共享。若每个 `ProcessVm` 都立即解除代码段和堆映射，就会重复释放相同物理帧。因此清理时先释放进程私有栈，仅当 `Arc::strong_count(page_table) == 1` 时，才由最后一个地址空间持有者清理共享堆、ELF 页面和低级页表。最顶层 P4 页由页表根对象的 `Drop` 释放。

3. 内核内存统计

Bootloader 加载内核 ELF 时记录所有映射页范围，并通过 `BootInfo` 传给内核。内核 `ProcessVm` 以这些范围初始化代码段统计，再加上内核栈得到内核进程的虚拟内存用量。

进程列表同时读取帧分配器状态，输出已使用、已回收、总帧数和物理内存占用比例。统计反映的是当前由分配器管理的可用物理区域，不包括固件保留区域。

4. 内核栈自动增长

内核栈在配置中保留 1 MiB 虚拟地址范围，但启动时只映射顶部 8 页，即 32 KiB。访问尚未映射、但仍位于预留栈范围内的地址时触发页故障，统一的栈增长逻辑分配并映射缺失页面。

用户栈页面使用 `PRESENT | WRITABLE | USER_ACCESSIBLE`，内核栈只使用 `PRESENT | WRITABLE`。若错误地给内核栈设置 `USER_ACCESSIBLE`，会破坏内核与用户地址空间的权限边界。

5. brk 与用户堆

`brk` 调整进程数据段末端地址。调用参数为 0 时查询当前 `break`；参数非 0 时尝试设置新 `break`。用户请求的 `break` 可以不是页对齐地址，但页面映射必须按 4 KiB 对齐，因此实现分别维护：

- 精确 `break`：用于系统调用返回值和字节级内存统计；
- 对齐后的映射末端：用于决定需要新增或解除哪些页面。

扩展堆时逐页分配物理帧并映射；失败时回滚本次已经映射的页面。缩小堆时解除不再覆盖的整页映射并回收物理帧。用户库通过 `sys_brk` 封装系统调用，并启用 `brk_alloc` 作为全局分配器。

五. 实验过程与实现

1. 实现帧分配与回收

在 `BootInfoFrameAllocator` 中加入 `recycled` 集合和统计接口。分配顺序为“回收帧优先，新帧兜底”，释放时还使用调试断言检查同一帧未被重复加入回收集合。

这种计数语义使运行时可以直接显示：

```
Frames : 281 used, 0 recycled, 3183 total
```

当进程退出后，释放的页面进入 `recycled`；后续创建进程时优先复用这些帧，不再单调消耗启动内存图中的新帧。

2. 记录并释放 ELF 映射

修改 ELF 模块，使 `load_elf` 返回各可加载段实际覆盖的 `PageRangeInclusive`。

`ProcessVm::load_elf_code` 保存范围并累计页数。退出时遍历范围：

1. 使用 `mapper` 解除虚拟页映射；
2. 获得原物理帧并交给帧回收器；
3. 刷新对应 TLB 项；
4. 最后清理不再使用的下级页表。

零长度 ELF 段不会产生页范围，避免页数计算下溢。

3. 完善 ProcessVm 生命周期

`ProcessVm` 的 `Drop` 调用统一的 `clean_up`，确保正常退出、加载失败或所有者提前返回时都能释放资源。清理顺序为私有栈、共享地址空间内容、低级页表、顶级页表根对象。

原有页表根析构中递归释放用户页的逻辑被移除，避免它与 `ProcessVm` 的区域清理重复释放同一物理帧。页表根析构现在只负责自己明确拥有的 P4 帧。

4. 添加内存显示

`Process` 的格式化实现和 `print_process_list` 增加 Memory 列，显示每个进程的 ELF、栈和堆总占用。进程列表末尾增加物理内存与帧状态，例如：

```
shell  47.99 K  Running
kernel 32.29 M  Ready
Memory : 1.10 MiB / 12.43 MiB (8.83%)
Frames : 281 used, 0 recycled, 3183 total
```

5. 传递内核页范围并启用内核栈增长

启动程序保存加载内核 ELF 时得到的映射范围，在 `BootInfo` 中新增 `kernel_pages`。内核进程据此初始化 `ProcessVm` 的代码区域。

`boot.conf` 将内核栈虚拟范围设为 1 MiB，并指定启动时只映射 8 页。内核页故障处理不再直接终止，而是先交给当前进程的 `ProcessVm` 判断故障地址是否位于可增长的栈范围内。

6. 实现 `brk` 完整调用链

系统调用链依次完成以下修改：

```
用户程序
-> yslib::sys_brk
-> Syscall::Brk = 12
-> syscall dispatcher
-> service::sys_brk
-> proc::brk
-> ProcessInner::brk
-> ProcessVm::brk
-> Heap::brk
```

内核入口使用 `VirtAddr::try_new` 拒绝非规范地址，并以 `usize::MAX` 表示失败。堆实现检查地址是否位于预设用户堆区间，扩展时具有失败回滚，缩小时回收整页。

用户库默认特性由 `kernel_alloc` 切换到 `brk_alloc`。同时删除旧的 `allocator.rs`，解决它与 `allocator/mod.rs` 同时存在导致的 Rust 模块冲突，并修正工作区依赖中的包名为实际的 `yslib`。

六. 测试与结果

1. 静态检查与构建

完成以下检查，均通过：

```
cargo check -p yyos_kernel --lib --offline
cargo check -p yslib --offline
cargo check -p yyos_boot --lib --offline
cargo check -p yyos_elf --offline
make target/x86_64-unknown-yyos/release
make target/x86_64-unknown-none/release/yyos_kernel
make target/x86_64-unknown-uefi/release/yyos_boot.efi
make build
git diff --check
```

用户程序 `shell` 、 `hello` 、 `dinner` 、 `fork` 、 `counter` 、 `fish` 、 `factorial` 和 `mq` 均能为自定义用户态目标成功构建。相关实验目录中不再存在未完成的 `FIXME/TODO` 。

2. QEMU 运行测试

系统可以通过 UEFI 正常启动。日志显示内核栈总预留大小为 1 MiB，初始映射 8 页；Shell 启动时 `brk` 分配器成功初始化。

执行 `stat` 能看到进程内存和物理帧统计。执行 `hello` 后，用户堆发生如下扩展：

```
Adjust heap break:
0x2000000000000 -> 0x200000001ff8
mapped end:
0x2000000000000 -> 0x200000002000
```

`hello` 正常输出并以 233 退出。退出日志出现顶级页表释放记录，Shell 正确收到退出码，系统未发生 panic、重复释放或页故障循环。QEMU 最终仅由测试命令的超时机制终止。

七. 问题与解决

1. 共享地址空间导致重复释放

最初若同时在 `ProcessVm` 清理和页表根析构中遍历用户页面，退出时会对同一帧执行两次回收。解决方法是明确所有权：`ProcessVm` 负责代码、栈、堆和下级页表，页表根对象只负责 P4；共享部分只由最后一个 `Arc` 持有者清理。

2. 用户栈与内核栈权限不同

直接复用用户栈增长函数会给内核栈加上 `USER_ACCESSIBLE`。为 `Stack` 增加 `user_access` 属性后，映射标志由栈类型决定，既复用了增长逻辑，也保持了页级权限隔离。

3. break 与页面边界不一致

`brk` 的接口按字节工作，而页表按页面工作。若只保存对齐地址，会向用户返回错误的 `break`；若完全按字节解除映射，又会错误释放仍被堆覆盖的页面。因此实现保存精确 `break`，并仅根据向上对齐后的边界增删页面。

4. 构建配置中的历史冲突

工程中同时存在 `allocator.rs` 与 `allocator/mod.rs`，`Rust` 无法确定模块来源；工作区又将实际包 `yslib` 写成了 `yylib`。删除过时模块文件并修正 `package` 名后，用户态自定义目标可以正常编译。

八. 思考题

1. 删除正在运行的 ELF 文件会发生什么

`Linux` 中运行程序的地址空间持有可执行文件对应 `inode` 和映射的引用。`unlink` 只删除目录项；只要仍有进程引用该 `inode`，文件数据不会立即回收，进程也可以继续运行。尚未调入的文件页仍可通过已有映射和页缓存读取。当最后一个文件描述符、映射和 `inode` 引用都释放后，存储空间才真正回收。访问不属于有效映射的地址仍会触发 `SIGSEGV`。

2. 为什么 `Arc::strong_count` 是关联函数

`Arc<T>` 实现 `Deref<Target = T>`。如果 `strong_count` 设计成普通方法，可能与 `T` 自身的同名方法在自动解引用中产生语义混淆。写成 `Arc::strong_count(&value)` 可以明确表示查询的是 `Arc` 控制块，而不是内部对象的方法，也不会占用 `T` 的方法命名空间。

3. 内核栈最少映射多少页

不存在对所有内核都固定的最小值。初始页面必须支撑从进入内核到页故障处理器、帧分配器、`GDT/IDT` 和日志设施都可用之前的最大栈深度。本实现中 8 页即 32 KiB 可以稳定启动。若

初始映射过小，内核可能在页故障处理能力尚未建立时再次缺页，进一步引发双重故障甚至三重故障重启。

4. mmap、munmap 与 mprotect

`mmap` 创建文件或匿名内存映射，`munmap` 删除映射，`mprotect` 修改映射页面的读、写、执行权限。下面的 Linux 示例把文件映射为共享可写内存并显式同步：

```
int fd = open("data.bin", O_RDWR);
ftruncate(fd, 4096);
char *p = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
               MAP_SHARED, fd, 0);
memcpy(p, "hello", 5);
msync(p, 4096, MS_SYNC);
mprotect(p, 4096, PROT_READ);
munmap(p, 4096);
close(fd);
```

对 `MAP_SHARED` 映射的写入会先修改内存页，随后由 `msync`、解除映射或内核回写机制同步到文件；`MAP_PRIVATE` 则使用写时复制，修改不会回写原文件。

九. 总结

本实验将上一阶段“能够分配页面”的实现扩展为具有完整生命周期的内存管理：物理帧可以回收复用，进程退出会释放 ELF、栈、堆和页表，内核栈可以按需增长，用户程序可以通过 `brk` 使用动态堆，同时系统能够观察进程和物理内存占用。

实现中最关键的是明确所有权边界。只有当每类页面都有唯一、可追踪的释放责任时，`fork`、进程退出和错误回滚才能同时正确。最终构建与 QEMU 实测均通过，实验任务要求的相关占位实现已经补全。

十. 参考资料

1. YatSenOS 实验七任务：<https://ysos.gzti.me/labs/0x07/tasks/>
2. Rust `alloc::sync::Arc` 文档。
3. x86_64 页表与页故障处理相关文档。
4. Linux `mmap(2)`、`munmap(2)`、`mprotect(2)` 手册。